CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 12
Network Flow, Min-Cut & Matching

One algorithmic idea that solves a surprising number of problems, once you learn to recognise them.

Saturday 12 December 2026 · 3 hours

PS4 released today (due Sat 26 Dec) · Project M5 (preliminary results) due today

Motivating problems

Throughput

A delivery network with road capacities (trucks/hour). How many trucks per hour can reach the port? *Which roads are the bottleneck?*

Assignment

300 interns, 120 projects; each intern is acceptable to some projects, and each project takes a limited number of interns. *Can everyone be placed?*

Impossibility

A sports league mid-season: can team X still finish first? *If not, can we give a short, checkable reason?*

DISCUSS · 3 MIN

What do these three problems have in common? What would a "certificate" that the answer is optimal (or impossible) look like?

BLOCK A · 00:10–01:00

Max-flow and min-cut

Definitions · residual graphs · Ford–Fulkerson · the theorem · Edmonds–Karp · Dinic

Flow networks, cuts, weak duality

A directed graph with capacities $c(u,v) \geq 0$, a source s and a sink t. A **flow** f satisfies

- **capacity:** $0 \leq f(u,v) \leq c(u,v)$;
- **conservation:** inflow = outflow at every $v \neq s, t$.

Value $|f|$ = the net flow out of s.

An s–t **cut** (S,T): $s \in S$, $t \in T$. Its capacity $c(S,T) = \sum_{u \in S, v \in T} c(u,v)$ counts only forward edges.

LEMMA (FLOW ACROSS ANY CUT)

For every flow f and every cut (S,T): $|f| = f(S,T) - f(T,S) \leq c(S,T)$.

PROOF

Sum conservation over the vertices of S: the net flow out of S equals the net flow out of s, which is $|f|$. The net flow out of S is $f(S,T) - f(T,S) \leq c(S,T) - 0$. ∎

So every cut is an **upper bound** on every flow: max-flow ≤ min-cut. The theorem says they are equal.

Residual graphs and augmenting paths

Residual graph G_f: for each edge (u, v),

- a forward residual edge with capacity $c(u, v) - f(u, v)$ (room to push more);
- a backward residual edge (v, u) with capacity $f(u, v)$ (the ability to **undo** flow).

An **augmenting path** is an s–t path in G_f. Pushing its bottleneck Δ along it keeps capacity and conservation, and increases $|f|$ by Δ.

WHY BACKWARD EDGES ARE ESSENTIAL

Greedy "push along any path with spare capacity, never undo" can get stuck below the optimum: an early path may use an edge that two later paths needed.

The backward residual edge lets a later augmentation **reroute** flow. That is what makes the method complete.

Ford–Fulkerson and the max-flow/min-cut theorem

```
FORD-FULKERSON(G, s, t)
  f = 0
  while G_f has an s–t path P:  augment f along P by its bottleneck
  return f
```

THEOREM (FORD & FULKERSON 1956)

The following are equivalent: (1) f is a maximum flow; (2) G_f has no augmenting path; (3) $|f| = c(S,T)$ for some cut (S,T). In particular, **max-flow = min-cut**.

PROOF

(1)⇒(2): an augmenting path would increase $|f|$. (3)⇒(1): weak duality. (2)⇒(3): let S = the vertices reachable from s in G_f, so $t \notin S$. Every edge $S \to T$ is **saturated** (otherwise its head would be reachable). Every edge $T \to S$ carries **zero** flow (otherwise its backward residual edge would make its tail reachable). So $|f| = f(S,T) - f(T,S) = c(S,T)$. ∎

Choosing the augmenting path: Edmonds–Karp and Dinic

Edmonds–Karp (1972): shortest augmenting path (BFS)

- **Lemma:** the BFS distance $d_f(s, v)$ never decreases across augmentations.
- Each edge becomes the bottleneck ("critical") at most $O(n)$ times: between two consecutive times, the distance of its tail grows by ≥ 2.
- So there are $O(nm)$ augmentations, each $O(m)$: $O(nm^2)$, independent of the capacities (polynomial).

Dinic (1970): blocking flows in the level graph

- Build the BFS *level graph* (edges from level i to $i + 1$).
- Find a **blocking flow** in it (DFS with current-arc pointers), then rebuild.
- The s–t distance increases every phase: $\leq n$ phases, $O(nm)$ each: $O(n^2m)$.
- Unit-capacity bipartite graphs: $O(m\sqrt{n})$ (= Hopcroft–Karp).

> The generic method is correct with *any* augmenting path. **Which** path is chosen decides whether the running time depends on the capacity values (pseudo-polynomial) or only on n and m (polynomial).

Exercise A: by hand

Network: $s \to a$ (10), $s \to b$ (5), $a \to b$ (15), $a \to c$ (4), $b \to d$ (8), $c \to t$ (10), $d \to c$ (6), $d \to t$ (5).

1. Run Edmonds–Karp. Show each augmenting path and its bottleneck.
2. Give the final residual graph's reachable set S and the minimum cut. Check that its capacity equals the flow.
3. Which single edge would you upgrade (+5 capacity) to increase the throughput the most? Justify with the cut.

PAIRS · 10 MIN

Then: if every capacity is multiplied by 1000, how many augmentations do FF (with an unlucky path rule) and Edmonds–Karp need?

BLOCK B · 01:10–02:00

Modelling with flow

Matching · the toolkit · project selection · elimination

Bipartite matching as flow

A bipartite graph $(L \cup R, E)$. Build: $s \to \ell$ (capacity 1) for $\ell \in L$; $\ell \to r$ (capacity 1) for $(\ell, r) \in E$; $r \to t$ (capacity 1).

CLAIM

Maximum matching size = maximum flow value.

PROOF (BOTH DIRECTIONS)

($\leq$) A matching M gives a flow of value $|M|$: one unit along $s \to \ell \to r \to t$ for each matched pair; capacities hold because the pairs are disjoint. ($\geq$) By integrality there is an integral maximum flow, so each edge carries 0 or 1. Unit capacity into each ℓ and out of each r means each vertex is used at most once, so the middle edges with flow 1 form a matching of the same size.

■

KÖNIG'S THEOREM (A COROLLARY)

In a bipartite graph, **max matching = min vertex cover**. A minimum cut (with capacities ∞ on the middle edges) corresponds to a vertex cover: $L \setminus S$ plus $R \cap S$.

In **general** graphs, maximum matching is still polynomial (Edmonds' blossom algorithm), but not via this simple flow model. And minimum vertex cover becomes NP-hard (L13).

The modelling toolkit

Feature of the problem	Construction
Several sources / sinks	a super-source $s^* \to s_i$ (capacity = the supply of s_i, or ∞); likewise a super-sink
Capacity on a **vertex**	split v into $v_{in} \to v_{out}$ with that capacity; incoming edges go to v_{in}, outgoing edges leave v_{out}
Undirected edge of capacity c	two directed edges $u \to v$, $v \to u$, each of capacity c
"Each item used at most k times"	capacity k on the item's source or sink edge
"Must never be separated" / a hard constraint	an edge of capacity ∞ (it can never be in a finite cut)
Lower bounds / demands	circulation with demands (KT 7.7): reduce to plain max-flow
Costs per unit of flow	min-cost flow (successive shortest paths with potentials: L10)

Min-cut as an optimisation tool: project selection

Projects with profits p_i (possibly negative = cost). Prerequisites: choosing i requires choosing j. Maximise the total profit of a prerequisite-closed set.

Network: $s \to i$ with capacity p_i if $p_i > 0$; $i \to t$ with capacity $-p_i$ if $p_i < 0$; $i \to j$ with capacity ∞ for each prerequisite.

Claim: for a minimum cut (S, T), the projects in $S \setminus \{s\}$ are an optimal selection, with profit $= \sum_{p_i>0} p_i - c(S,T)$.

WHY

A finite cut cannot cut an ∞ edge, so S is closed under prerequisites. Any closed set A gives a finite cut $(\{s\} \cup A,\ \text{rest})$.

Its capacity = (the profits of positive projects **not** chosen) + (the costs of negative projects chosen) = $\sum_{p_i>0} p_i - \text{profit}(A)$.

So minimising the cut maximises the profit. A bijection between closed sets and finite cuts.

■

Certificates of impossibility: baseball elimination

Team x has w_x wins and r_x games left. Assume x wins all of them: $W = w_x + r_x$. Is there an outcome of the remaining games among the *other* teams in which no team exceeds W?

Network: $s \to$ (game node $\{i, j\}$) with capacity = the number of games left between i and j; game $\to i$ and game $\to j$ with ∞; $i \to t$ with capacity $W - w_i$.

x can still win iff the max flow saturates all game edges.

THE CERTIFICATE

If x is eliminated, the min cut yields a set T' of teams with

$$\frac{w(T') + g(T')}{|T'|} > W,$$

where $g(T')$ is the number of games left among them: their **average** number of wins must exceed W, so some team exceeds it. A one-line proof that anyone can check, found by the algorithm.

Exercise B: model it (and prove both directions)

1. **Dinner seating.** Families i of size a_i, tables j with b_j seats. No two members of the same family may sit at the same table. Is a seating possible?
2. **Tutoring.** Students need tutoring in subjects; tutors can teach some subjects and have at most k_t hours. Each student needs one hour per requested subject, possibly from different tutors. Maximise the number of hours scheduled.
3. **Evacuation.** A building graph with corridor capacities (people/min) and staircase *node* capacities; several exits. Maximum evacuation rate?

Groups · 12 min · for each: the network, and one sentence per direction of the correspondence

BLOCK C · 02:10–02:55

Evidence and recognition

Path rules matter · matching at scale · flow or not?

Experiments: path choice and scale

1. The classic bad network (4 nodes, capacities C, a middle edge of capacity 1)

C	FF, longest-path rule: augmentations	Edmonds–Karp
1,000	2,000	2
100,000	200,000	2
1,000,000	2,000,000	2

2. Maximum bipartite matching, $n + n$ vertices, degree d (ms)

n	d	FF (DFS)	Edmonds–Karp	Dinic
2,000	3	14.7	35.5	2.1
20,000	3	2,165	6,753	32.2
20,000	10	1,700	10,336	42.7
100,000	**10**	**–**	**–**	**635**

`Student_Resources/flow_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 5 MIN

1. Table 1: what does the number of augmentations depend on for each rule? Relate it to input size.

2. Table 2: Edmonds–Karp has the better worst-case bound, yet it loses to DFS-FF. Why?

3. Why is Dinic so much faster on matching?

Exercise C: flow or not?

For each problem: can it be solved exactly with (a polynomial number of) max-flow computations? If yes, give the model. If not, explain what breaks.

1. Maximum number of **vertex**-disjoint s–t paths in a directed graph.
2. A Hamiltonian path from s to t.
3. Assign workers to shifts where each worker must work **exactly** 3 shifts or none at all.
4. The minimum number of edges to delete so that G (undirected) becomes disconnected (global min cut).
5. A maximum matching in a general (non-bipartite) graph, via the flow network above with $L = R = V$.

Groups · 12 min

Synthesis

1. Every cut bounds every flow (weak duality). **Max-flow = min-cut**, and the proof builds the cut from the residual graph.
2. Ford–Fulkerson is correct with any augmenting path. The **path rule** decides pseudo-polynomial vs. polynomial: FF $O(m|f^*|)$, Edmonds–Karp $O(nm^2)$, Dinic $O(n^2m)$ and $O(m\sqrt{n})$ on unit bipartite graphs.
3. **Integrality** turns flows into combinatorial objects (matchings, assignments).
4. Modelling = the unit, the capacities (edges, vertices, uses), conservation, and a proof in **both directions**. Min-cuts answer optimisation (project selection) and impossibility (elimination) questions.
5. Some problems are not flow problems: Hamiltonian paths, "all or nothing" constraints. That is next week.

NEXT WEEK · L13 (SAT 19 DEC)

NP-completeness: certificates, reductions (and their direction!), SAT → 3-SAT → Independent Set → Vertex Cover → Clique; then exhaustive search, backtracking and branch-and-bound done properly.

Read: KT 8.1–8.4; CLRS 4e Ch. 34. **PS4** due Sat 26 Dec; **final report** due Thu 24 Dec.